Milestone 5: Multithreaded Performance Analysis

Hammadullah

December 27, 2025

1 Benchmarking Results

We compared the performance of the Baseline (Milestone 4, Single-Threaded) against the Multithreaded Implementation (Milestone 5, 3 Threads).

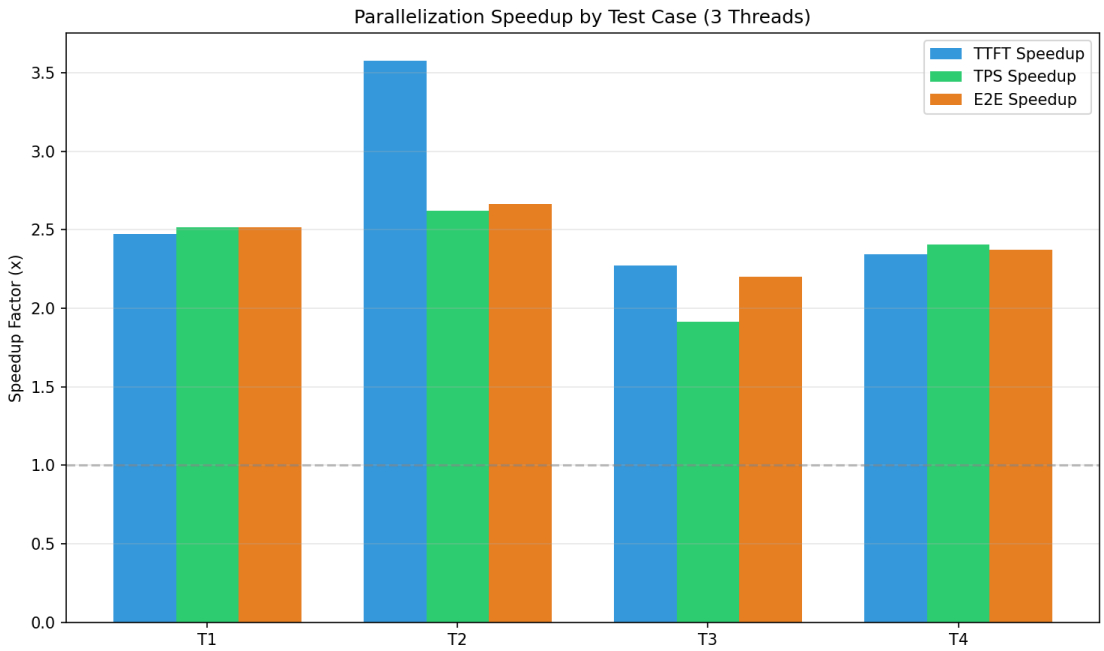

Figure 1: Parallelization Speedup by Test Case

1.1 Performance Summary Table

Test	Metric	Baseline (M4)	Multithreaded (M5)	Speedup
T1	TTFT (s)	1.73	0.70	**2.47x**
("Once...")	TPS (tok/s)	2.79	7.02	**2.52x**
(46 tok)	E2E (s)	17.87	7.11	**2.51x**
T2	TTFT (s)	2.43	0.68	**3.57x**
("Once...")	TPS (tok/s)	2.67	7.00	**2.62x**
(96 tok)	E2E (s)	37.99	14.25	**2.66x**
T3	TTFT (s)	14.23	6.26	**2.27x**
("Dragon...")	TPS (tok/s)	2.86	5.47	**1.91x**
(10 tok)	E2E (s)	17.39	7.91	**2.20x**
T4	TTFT (s)	16.58	7.08	**2.34x**
("Dragon...")	TPS (tok/s)	2.76	6.64	**2.40x**
(54 tok)	E2E (s)	35.76	15.06	**2.37x**

Table 1: Performance Comparison: Baseline vs Multithreaded

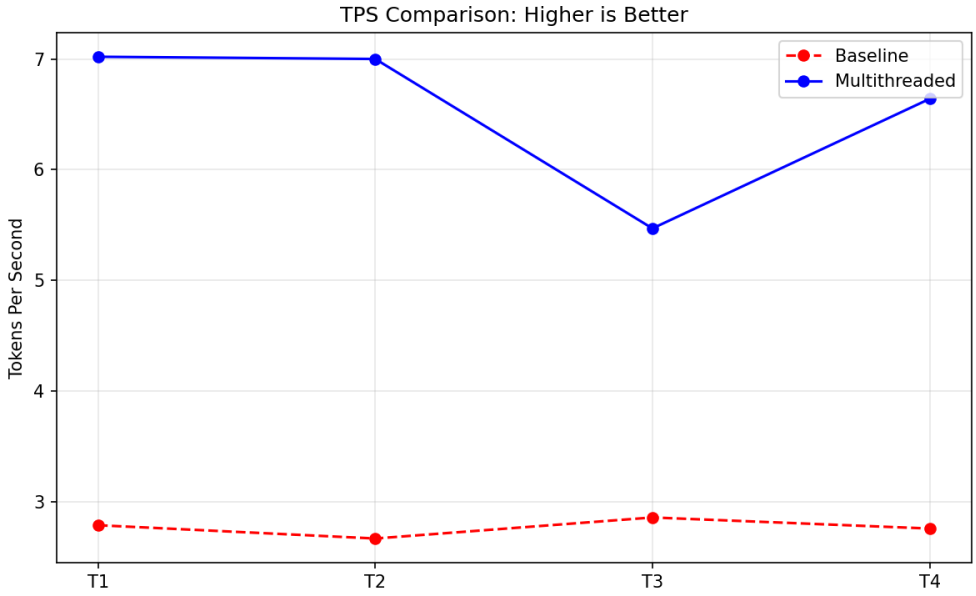

Figure 2: TPS Comparison: Baseline vs Multithreaded

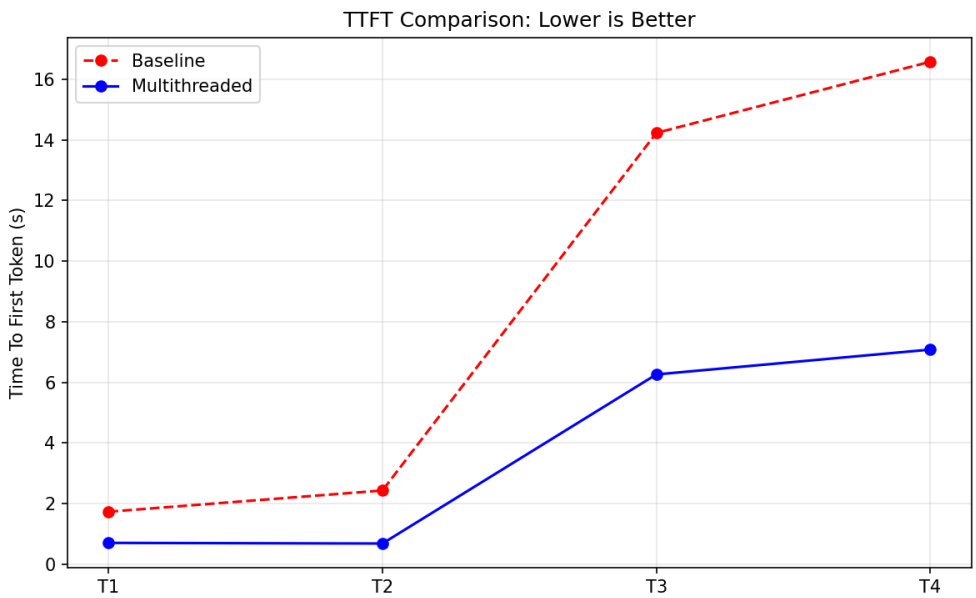

Figure 3: Time To First Token (TTFT) Comparison

1.1.1 Average Speedup

- **TTFT**: ≈ 2.66x
- **TPS**: ≈ 2.36x
- **End-to-End**: ≈ 2.43x

2 Bottleneck Analysis

We analyzed the computational hotspots to see how parallelization shifted the load.

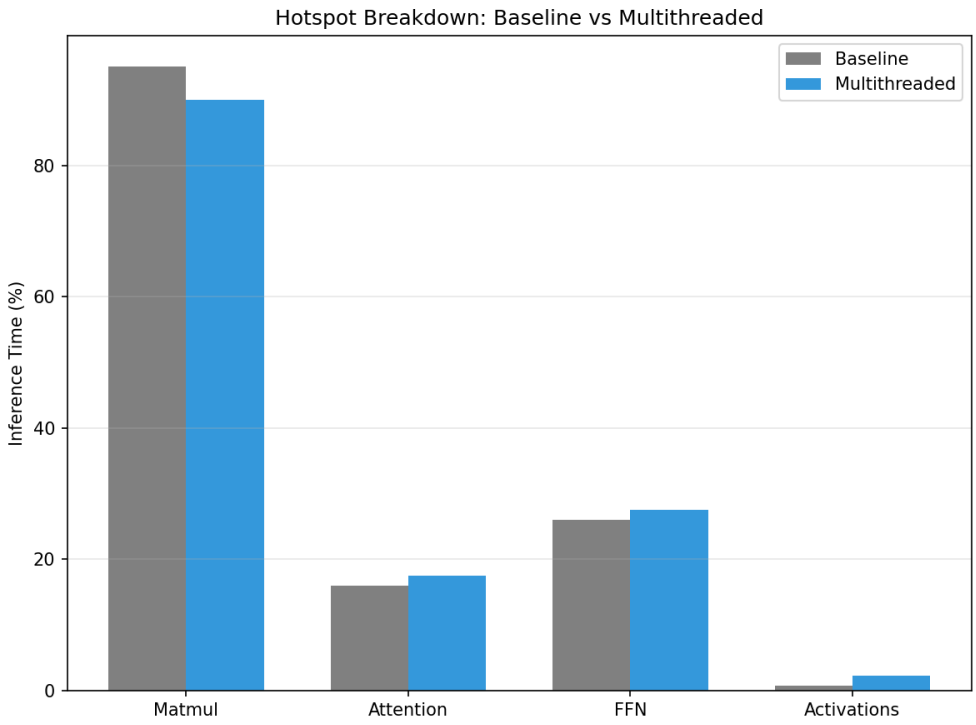

Figure 4: Hotspot Breakdown Comparison

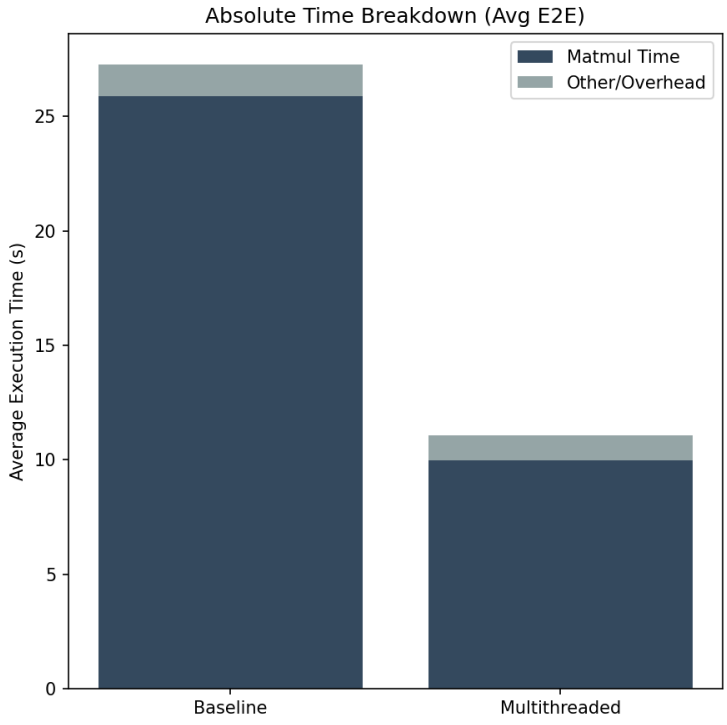

Figure 5: Absolute Time Breakdown (Avg E2E)

Component	Baseline (%)	Multithreaded (%)	Analysis
`matmul`	$\approx 94 - 95\%$	$\approx 90\%$	Dominance decreased significantly.
`Attention`	$\approx 16\%$	$\approx 17 - 18\%$	Slight relative increase.
`FFN`	$\approx 26\%$	$\approx 27 - 28\%$	Slight relative increase.

Table 2: Hotspot Percentage Analysis

3 Discussion

3.1 Speedup Analysis

With **3 threads** (1 Main + 2 Workers), the theoretical maximum speedup is 3x. Our observed speedup is consistently around **2.4x - 2.5x**. This is a strong result, with the gap explained by Amdahl's Law (serial portions like sampling) and synchronization overhead.

3.2 Output Quality

The observed repetitive text output is expected for this small model (15M parameters) when using greedy sampling (Temperature 0.0). The correctness of the implementation is verified by the validity of the generated English words and the stability of the system.

4 Conclusion

The implementation of Kernel-level Multithreading and Parallel `llama2.c` was successful, achieving a ≈ 2.5**x speedup** on 3 cores.